RUNTIME PARALLEL – STUDENT NOTES

PREVIOUS SESSION RECAP

1. Prompt | Model | Parser
2. invoke(), batch(), stream() methods
3. RunnableSequence for sequential workflows

Key Point:
If tasks are dependent → use Sequence

Example:
If the output of one task is required as input for another, then use RunnableSequence.

WHEN TASKS ARE INDEPENDENT

Example:

Input: Artificial Intelligence

Required outputs:

1. Blog
2. Social Media Post
3. Short Summary

These tasks are independent (no dependency between them).
So, we use Parallel Execution.

REAL-LIFE ANALOGY

Birthday preparation:

Person 1 → Orders cake
Person 2 → Decorates hall
Person 3 → Invites guests

All tasks can happen at the same time.
No one needs to wait for others.

This is called Parallel Work.

WHAT IS PARALLEL EXECUTION?

Parallel execution means multiple independent tasks can run at the same time using the same input, without waiting for each other.

IMPORTANT CONCEPT

DEPENDENT TASKS → SEQUENCE
INDEPENDENT TASKS → PARALLEL

WHAT IS RUNNABLEPARALLEL?

RunnableParallel is used to run multiple tasks (runnables) from the same input and return all outputs together.

One input → Multiple outputs

Output is returned as a dictionary.

BASIC SYNTAX

```
from langchain_core.runnables import RunnableParallel

parallel = RunnableParallel(
task1 = ...,
task2 = ...
)
```

Each key represents one output branch.

SIMPLE EXAMPLE (WITHOUT AI)

Input: 5

Operations:
add_ten → x + 10
double → x * 2
square → x * x

Output:
{
'add_ten': 15,
'double': 10,
'square': 25
}

Same input is given to all branches.

WHY BRANCH NAMES ARE IMPORTANT

Example:

parallel = RunnableParallel(
blog = blog_chain,
summary = summary_chain
)

Access results like:
result["blog"]
result["summary"]

Branch names help identify outputs easily.

FIRST AI PARALLEL APPLICATION

Goal:
From the same topic generate:

- Joke
- Motivation

Two independent chains are created.
Both receive the same input.

Output:
{

"joke": "...",
"motivation": "..."
}

PRACTICAL USE CASE

Input: Artificial Intelligence

Required:

1. Blog
2. Social Media Post
3. Summary

This is a perfect use case for RunnableParallel.

STEPS

1. Create prompts
    - Blog prompt
    - Social prompt
    - Summary prompt
2. Create chains
    - blog_chain
    - social_chain
    - summary_chain
3. Combine using RunnableParallel

parallel = RunnableParallel(
blog = blog_chain,
social_post = social_chain,
summary = summary_chain
)

EXECUTION

result = parallel.invoke({
"topic": "Artificial Intelligence"
})

All branches receive the same input.

COMBINING SEQUENCE AND PARALLEL (VERY IMPORTANT)

Real-world applications use both.

Example:

Step 1: Generate explanation

Step 2: From that explanation generate in parallel:

- Summary
- Quiz
- Social Post

FINAL CONCEPT

Sequence + Parallel can be combined for building real applications.

KEY TAKEAWAYS

- Parallel is used for independent tasks
- Same input can produce multiple outputs
- Output is a dictionary
- Faster execution
- Can be combined with sequence